Problem 1

1-(a) $h(n) = O(\log_2 n) : \exists (c_2>0, n_0>0)$ s.t. $h(n) \le c_2 \log_2 n = c_2 \log_2^3 \log_3 n$

let $c_2' = c_2 \log_2^3$, $n_0' = n_0$. Then (c_2', n_0') witnesses

$h(n) \le c_2' \log_3 n \Rightarrow h(n) = O(\log_2 n) = O(\log_3 n)$

1-(b) For asymptotically non-negative functions $f(n)$ and $g(n)$,

if $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \Rightarrow |\frac{f(n)}{g(n)} - 0| < \epsilon$, $0 \le \frac{f(n)}{g(n)} < \epsilon$, $f(n) = \epsilon g(n)$

let $c = \epsilon$ and $n_0 = n$, we have $f(n) < cg(n)\ \forall n \ge n_0$, which matches the definition of $f(n) = o g(n)$.

$\because \lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} (\frac{2^n}{3^n}) = 0 \quad \therefore f(n) = o g(n) \Rightarrow O(f(n)) \subsetneq O(g(n))$

Prove $f(n) = o g(n) \Rightarrow O(f(n)) \subsetneq O(g(n))$:

let $h(n) = O(f(n))$, By definition $\exists (c_1>0, n_0>0)$ s.t. $h(n) \le c_1 f(n)$

we known that $f(n) = o g(n)$

$\Rightarrow \exists (c>0, n_1>0)$ s.t. $f(n) < cg(n)\ \forall c>0, n \ge n_1$

Hence, choose $c=1$, we have $f(n) < g(n)$

Hence, we have $h(n) \le c_1 f(n) < c_1 g(n) \Rightarrow h(n) \in O(g(n))$

$\Rightarrow O(f(n)) \subseteq O(g(n))$

Next, we want to prove that $O(f(n)) \ne O(g(n))$:

since $g(n) \le 1 \cdot g(n) \Rightarrow g(n) \in O(g(n))$

assume $g(n) \in O(f(n)) \Rightarrow \exists (c_2>0, n_2>0)$ s.t. $g(n) \le c_2 f(n) \Rightarrow \frac{1}{c_2} g(n) \le f(n)$

we know $f(n) = o g(n)$, if we choose the constant to be $\frac{1}{2c_2}$, then we have: $f(n) < \frac{1}{2c_2} g(n) \Rightarrow g(n) \le c_2 f(n) < \frac{1}{2} g(n)$, since $g(n) \ge 0$, $\frac{1}{2} g(n)$ will never be larger than $g(n) \Rightarrow$ the initial assumption $g(n) \in O(g(n)) \in O(f(n))$ is false

$\Rightarrow$ it must be the case that $O(f(n)) \subsetneq O(g(n))$

1- (c) $n^n = \underbrace{n \times n \times \cdots \times n}_{n}$, $n! = \underbrace{1 \times 2 \times 3 \times \cdots \times n}_{n}$

$\forall n > 1$, $n^n > n! \Rightarrow \lim_{n\to\infty} \frac{g(n)}{f(n)} = \lim_{n\to\infty} \frac{n!}{n^n} = 0$

$\Rightarrow f(n) = \omega(g(n)) \Rightarrow g(n) = o(f(n)) \Rightarrow O(g(n)) \subseteq O(f(n))$

2. Consider $m \geq m_0$ and fixing m constant, $n \geq n_0$

$\exists (c_2 > 0, n_0 > 0, m_0 > 0)$ s.t. $mn \leq c_2 n \Rightarrow mn = O(n)$

$\exists (c_2' > 0, n_0' > 0, m_0' > 0)$ s.t. $m \log_2 n \leq c_2' \log_2 n \Rightarrow m \log_2 n = O(\log n)$

$\Rightarrow O(g(m,n)) \subseteq O(f(m,n))$

Consider $m \geq m_0$, $n \geq n_0$ and $m = n$,

$f(m,n) = 2n$, $g(m,n) = n \log_2 n$

$\exists (c_2 > 0, n_0 > 0)$ s.t. $2n \leq c_2 n \Rightarrow f(m,n) = O(n)$

$\exists (c_2' > 0, n_0' > 0)$ s.t. $n \log_2 n \leq c_2 n \log_2 n \Rightarrow g(m,n) = O(n \log n)$

$\Rightarrow O(f(m,n)) \subseteq O(g(m,n))$

Since $f(m,n)$ bounds $g(m,n)$ in some paths but not all, the relationship between them is none of the above.

3-(a)

		cost	reps
	DISCREET-SORT(arr)		
1	$i \leftarrow 0$	d_1	1
2	$n \leftarrow length(arr)$	d_2	1
3	**while** $i < n$	d_3	$n+1$
4	**if** $i = 0$ or $arr[i-1] \leq arr[i]$	d_4	n
5	$i \leftarrow i + 1$	d_5	n
6	**else**		
7	SWAP($arr[i], arr[i-1]$)	d_7	0
8	$i \leftarrow i - 1$	d_8	0

Best case: the array is in a sorted order

$T(n) = d_1 + d_2 + d_3(n+1) + d_4 n + d_5 n$ $\Rightarrow$ linear to n; $T(n) = \Theta(n)$

3-(b)

		cost	reps
	DISCREET-SORT(arr)		
1	$i \leftarrow 0$	d_1	1
2	$n \leftarrow length(arr)$	d_2	1
3	**while** $i < n$	d_3	
4	**if** $i = 0$ or $arr[i-1] \leq arr[i]$	d_4	r_4
5	$i \leftarrow i + 1$	d_5	r_5
6	**else**	d_6	0
7	SWAP($arr[i], arr[i-1]$)	d_7	r_7
8	$i \leftarrow i - 1$	d_8	r_8

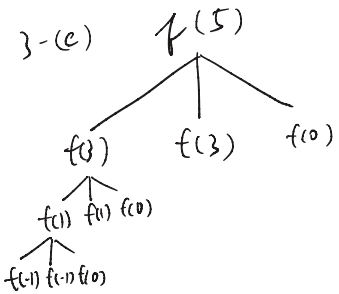
$r_4 = r_5 + r_8$, $r_5 = \sum_{\bar{i}=1}^{n-1} \bar{i} + n$, $r_7 = \sum_{\bar{i}=1}^{n-1} \bar{i}$, $r_8 = \sum_{\bar{i}=1}^{n-1} \bar{i}$

Worse case: the array is reversely sorted. Under this situation, when $\bar{i}$ moves one step forward, it will have to go all the way back to the front and back to its current place before making another step forward. The total backward steps is $\sum_{\bar{i}=1}^{n-1} \bar{i} = \frac{n(n-1)}{2}$, the total forward steps is $\sum_{\bar{i}=1}^{n-1} \bar{i} + n = \frac{n(n+1)}{2}$, and the total

steps for the while loop will be $\frac{n(n-1)}{2} + \frac{n(n+1)}{2} + 1$

Hence, the time complexity will be $\Theta(n^2)$.

3-(c)



for every number n,

when n is odd number, $n = 2k-1$, $k \in \mathbb{N}$, and k is the number of subtractions needed to make $n \le 0$. When n is even, $n = 2k$, $k \in \mathbb{N}$, and k is the number of subtractions needed to make $n \le 0$. k will be the total depth (layer) of the tree, and a tree will have $2^k = 2^{\frac{n}{2}}$ (or $2^{\frac{n+1}{2}}$ for odd numbers).

Hence, the overall time complexity will be $O(2^{\frac{n}{2}})$

worst-case

4. let $q = 1+x$, $q^n = (1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$, take the term where $k = p+1$: $q^n > \binom{n}{p+1} x^{p+1} = \frac{n(n-1)\cdots(n-p)}{(p+1)!} x^{p+1}$

let $c = \frac{x^{p+1}}{(p+1)!}$ and $n(n-1)\cdots(n-p) = n^{p+1} + c_p n^p + c_{p-1} n^{p-1} + \cdots + p > n^p$

$q^n > c\,(n^{p+1} + c_p n^p + \cdots + p) = cn^{p+1} + c \cdot c_p n^p + \cdots + cp > c \cdot c_p n^p$

$n^p < \frac{1}{c \cdot c_p} q^n$, let $c_2 = \frac{1}{c \cdot c_p}$, $\exists (c_2 > 0, n_0 > 0)$ s.t. $n^p \le c_2 q^n$

$\forall n \ge n_0 \Rightarrow n^p = O(q^n)$

Problem 1

5.

```
left = 1
right = K
result = 0
while left <= right
   totalDays = 0
   mid = (left + right) / 2
   for i = 1 to n
      totalDays += (piles[i] + mid - 1) / mid
   if totalDays > h
      left = mid + 1
   else
      right = mid - 1
      result = mid

return result
```

By using binary search to search from 1 to K, we can achieve a time complexity of O(logK) for the searching process. Noted that since the optimal k will never be larger than K, we do not have to consider k > K. in the while loop, we use a for loop from 1 to n to find the sum of the days taken given k, which is the "mid" in this context. The days spent on a single pile is calculated by "(piles[i] + mid - 1) / 2", which gives us the ceiling of the result. Since the total days taken will decrease as we increase k, the number of snacks eaten every day, we will cut off the lower portion when "totalDays" exceeds h, the limit, and cut off the upper portion vice versa. Then we store the value in result and return it in the end. Since all the other operations inside the loops are of O(1) time complexity, the while loop is of O(logK), and the for loop is of O(n) time complexity, we can achieve an overall time complexity of O(nlogK).

Problem 2

1.

```
linkedList-Insertion-Sort(head)
   if head == NIL or head.next == NIL
      return head
   dummy = new Node()
   dummy.next = head
   lastSorted = head
   curr = head.next
   while curr != NIL
      if lastSorted.value <= curr.value
         lastSorted = lastSorted.next
      else
         comp = dummy
         lastSorted.next = curr.next
         while curr.value > comp.next.value
            comp = comp.next
         curr.next = comp.next
         comp.next = curr

      curr = lastSorted.next

   return dummy.next
```

2.

```
findDiffNum(array[], m)
   check = 0
   for i = 1 to 2m - 1
```

```
        check = check ^ array[i]


    return check
```

Since every integer from 1 to m appears twice, we can use XOR to cancel them out. The operation order does not matter in XOR, so we initialize "check" to 0 and XOR it with every element in the array. Since the elements that appear twice cancel each other out, the one that's left in the end is the element that only appears once. This algorithm only creates an int variable, taking O(1) space, and it only scan through the array once with only the XOR operation, taking O(m) time.

3.

```
linkedList-reorder(head)
    slow = head
    fast = head
    prevSlow = NIL
    while(fast.next != NIL)
        prevSlow = slow
        slow = slow.next
        if(fast.next.next != NIL)
            fast = fast.next.next
        else
            fast = fast.next
    prevSlow.next = NIL


    prev = NIL
    curr = slow
    while(curr != NIL)
```

```
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next

    currLeft = head
    currRight = prev
    while(currLeft != NIL && currRight != NIL)
        nextLeft = currLeft.next
        nextRight = currRight.next

        currLeft.next = currRight
        currLeft = nextLeft

        currRight.next = currLeft
        currRight = nextRight

    return head
```

For the first chunk (finding the middle), since the fast pointer moves twice as fast as the slow pointer, so when the fast pointer reaches the end, the slow one will be at the middle. Then in the second chunk, the algorithm reverses the second half of the list and sever the it from the first half. The last chunk links the nodes across the two halfs, reordering the nodes as expected. For a linked list with 2m nodes, finding the middle takes m steps as the fast pointer travels 2m nodes. Reversing the second half takes m steps. Merging the two halves takes 2m steps. Hence, the total time complexity is O(m) + O(m) + O(m) = O(m). As for the space complexity, this algorithm only set up a fixed number of pointers without dynamically allocating memories, so the space complexity remains O(1).

4.

In this context, 6, 7, 42 will be used to represent the multiple of 6, 7, 42, respectively, since the logic and the result

will not be affected. The pseudocode is wrong. For instance, if the input array is [6, 7, 42], there are m = 2 6s and k = 2 7s, the valid output will be [6, 42, 7], which places the 6s in the front and 7s in the back, making 42s in the middle. Yet, the output of this program will be [42, 6, 7] instead of [6, 42, 7]. The reason is that after the first loop, the program correctly places 7s to the back of the array, but in the second loop, the program incorrectly moves 42s to the front of the array, leaving 6s in the middle, while the output should be the opposite. This program will work only when there's no 42s in the input array.

5.

```
matrix_rotation(head)
  //find the head of every columns
  colHeadNodes[n]
  colCurrNodes[n]
  curr = head
  for col = 1 to n
    colHeadNodes[col] = curr
    colCurrNodes[col] = curr
    curr = curr.next

  //link by columns
  while curr != NIL
    for col = 1 to n
      colCurrNodes[col].next = curr
      colCurrNodes[col] = curr
      curr = curr.next
```

```
//link the last node of every column to the head of the previous column
for col = n to 2
    colCurrNodes[col].next = colHeadNodes[col - 1]
colCurrNodes[1].next = NIL

//reallocate the head of the list
head = colHeadNodes[n]

return head
```

A 90-degree counter-clockwise rotation transforms the first row to the first column, and the last column to the first row. First we set the every node in the first row to be the head of each column, then we connect the nodes in the same column. The final step is to connect the last node in every column to the first node of the previous column. By changing the direction of the links, the algorithm can mimic the rotation transformation. For time complexity, the algorithm checks every node in the list once, taking O(mn) time. Then it connect the last node in a column to the first node in the previous column. This step takes O(n) time since it only traverses through the last row of the original matrix. The overall time complexity is O(mn) + O(n) = O(mn). For space complexity, the algorithm only dynamically allocates two arrays of size n to store the head and the current node of each column, resulting in a space complexity of O(n).